

WINDOW FUNCTIONS

Chuẩn bị

```
CREATE TABLE employees (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(50),  
    department VARCHAR(50),  
    salary INT,  
    hire_date DATE  
);  
  
INSERT INTO employees (name, department, salary, hire_date) VALUES  
( 'Alice', 'IT', 800, '2020-01-15' ),  
( 'Bob', 'IT', 1200, '2019-03-12' ),  
( 'Charlie', 'IT', 1000, '2021-07-19' ),  
( 'David', 'HR', 1500, '2018-05-20' ),  
( 'Eve', 'HR', 1500, '2020-06-01' ),  
( 'Quang', 'HR', 1400, '2020-07-01' ),  
( 'Tuan', 'HR', 1600, '2020-07-01' ),  
( 'Frank', 'Finance', 1100, '2021-01-10' ),  
( 'Grace', 'Finance', 1300, '2019-09-25' );
```

Window Functions

- ❖ Window functions trong MySQL là các hàm phân tích cho phép thực hiện các phép tính **trên một tập hợp các hàng có liên quan đến hàng hiện tại** mà không gộp các hàng thành một kết quả duy nhất như các hàm tổng hợp thông thường.
- ❖ Đặc điểm
 - Được thêm vào MySQL từ phiên bản 8.0
 - Giữ nguyên các hàng riêng biệt trong kết quả (*không gộp như **GROUP BY***)
 - Sử dụng mệnh đề **OVER()** để xác định cách phân vùng và sắp xếp dữ liệu

Window Functions

```
window_function_name(expression) OVER (  
    [PARTITION BY column_list]  
    [ORDER BY column_list]  
    [frame_clause]  
)
```

PARTITION BY: Chia dữ liệu thành các nhóm (partitions) để áp dụng hàm window độc lập

ORDER BY: Xác định thứ tự sắp xếp các hàng trong mỗi partition

Frame clause: Xác định tập hợp các hàng trong partition để tính toán (ROWS hoặc RANGE)

Ví dụ

```
SELECT
    name,    department,    salary,
    COUNT(*) OVER (PARTITION BY department) AS team_size
FROM employees;
```

#	id	name	department	salary	hire_date
1	8	Alice	IT	800	2020-01-15
2	9	Bob	IT	1200	2019-03-12
3	10	Charlie	IT	1000	2021-07-19
4	11	David	HR	1500	2018-05-20
5	12	Eve	HR	1500	2020-06-01
6	13	Quang	HR	1400	2020-07-01
7	14	Tuan	HR	1600	2020-07-01
8	15	Frank	Finance	1100	2021-01-10
9	16	Grace	Finance	1300	2019-09-25



name	department	salary	team_size
Frank	Finance	1100	2
Grace	Finance	1300	2
David	HR	1500	4
Eve	HR	1500	4
Quang	HR	1400	4
Tuan	HR	1600	4
Alice	IT	800	3
Bob	IT	1200	3
Charlie	IT	1000	3

ROW_NUMBER()

```
SELECT
    name,    department,    salary,
    ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS row_num
FROM employees;
```

#	id	name	department	salary	hire_date
1	8	Alice	IT	800	2020-01-15
2	9	Bob	IT	1200	2019-03-12
3	10	Charlie	IT	1000	2021-07-19
4	11	David	HR	1500	2018-05-20
5	12	Eve	HR	1500	2020-06-01
6	13	Quang	HR	1400	2020-07-01
7	14	Tuan	HR	1600	2020-07-01
8	15	Frank	Finance	1100	2021-01-10
9	16	Grace	Finance	1300	2019-09-25

#	name	department	salary	row_num
1	Grace	Finance	1300	1
2	Frank	Finance	1100	2
3	Tuan	HR	1600	1
4	David	HR	1500	2
5	Eve	HR	1500	3
6	Quang	HR	1400	4
7	Bob	IT	1200	1
8	Charlie	IT	1000	2
9	Alice	IT	800	3

RANK()

```
SELECT
    name,    department,    salary,
    RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank_in_dept
FROM employees;
```

#	id	name	department	salary	hire_date
1	8	Alice	IT	800	2020-01-15
2	9	Bob	IT	1200	2019-03-12
3	10	Charlie	IT	1000	2021-07-19
4	11	David	HR	1500	2018-05-20
5	12	Eve	HR	1500	2020-06-01
6	13	Quang	HR	1400	2020-07-01
7	14	Tuan	HR	1600	2020-07-01
8	15	Frank	Finance	1100	2021-01-10
9	16	Grace	Finance	1300	2019-09-25

#	name	department	salary	rank_in_dept
1	Grace	Finance	1300	1
2	Frank	Finance	1100	2
3	Tuan	HR	1600	1
4	David	HR	1500	2
5	Eve	HR	1500	2
6	Quang	HR	1400	4
7	Bob	IT	1200	1
8	Charlie	IT	1000	2
9	Alice	IT	800	3

DENSE_RANK()

```
SELECT
    name,    department,    salary,
    DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank_in_dept
FROM employees;
```

#	id	name	department	salary	hire_date
1	8	Alice	IT	800	2020-01-15
2	9	Bob	IT	1200	2019-03-12
3	10	Charlie	IT	1000	2021-07-19
4	11	David	HR	1500	2018-05-20
5	12	Eve	HR	1500	2020-06-01
6	13	Quang	HR	1400	2020-07-01
7	14	Tuan	HR	1600	2020-07-01
8	15	Frank	Finance	1100	2021-01-10
9	16	Grace	Finance	1300	2019-09-25

#	name	department	salary	denserank
1	Grace	Finance	1300	1
2	Frank	Finance	1100	2
3	Tuan	HR	1600	1
4	David	HR	1500	2
5	Eve	HR	1500	2
6	Quang	HR	1400	3
7	Bob	IT	1200	1
8	Charlie	IT	1000	2
9	Alice	IT	800	3

LAG() LEAD()

```
SELECT
    name,    department,    salary,
    LAG(salary) OVER (PARTITION BY department ORDER BY salary DESC) AS previous_salary,
    LEAD(salary) OVER (PARTITION BY department ORDER BY salary DESC) AS next_salary
FROM employees;
```

#	id	name	department	salary	hire_date
1	8	Alice	IT	800	2020-01-15
2	9	Bob	IT	1200	2019-03-12
3	10	Charlie	IT	1000	2021-07-19
4	11	David	HR	1500	2018-05-20
5	12	Eve	HR	1500	2020-06-01
6	13	Quang	HR	1400	2020-07-01
7	14	Tuan	HR	1600	2020-07-01
8	15	Frank	Finance	1100	2021-01-10
9	16	Grace	Finance	1300	2019-09-25

name	department	salary	previous_salary	next_salary
Grace	Finance	1300	NULL	1100
Frank	Finance	1100	1300	NULL
Tuan	HR	1600	NULL	1500
David	HR	1500	1600	1500
Eve	HR	1500	1500	1400
Quang	HR	1400	1500	NULL
Bob	IT	1200	NULL	1000
Charlie	IT	1000	1200	800
Alice	IT	800	1000	NULL

Frame Clause

Chuẩn bị

```
CREATE TABLE sales (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    employee VARCHAR(50),  
    sale_date DATE,  
    amount INT  
);  
  
INSERT INTO sales (employee, sale_date, amount) VALUES  
('Alice', '2024-01-01', 100),  
('Alice', '2024-01-02', 200),  
('Alice', '2024-01-03', 300),  
('Alice', '2024-01-04', 400),  
('Bob', '2024-01-01', 150),  
('Bob', '2024-01-02', 250),  
('Bob', '2024-01-03', 350);
```

Window Functions

[PARTITION BY ...]

[ORDER BY ...]

[ROWS | RANGE BETWEEN ... AND ...]

- ❖ **FRAME_CLAUSE** giúp xác định phạm vi (window frame) mà hàm cửa sổ sẽ hoạt động
- ❖ Đặc biệt hữu ích khi dùng với hàm như SUM(), AVG(), FIRST_VALUE(), LAST_VALUE().

ROWS - Frame Clause

[PARTITION BY ...]

[ORDER BY ...]

[ROWS BETWEEN ... AND ...]

❖ ROW

- UNBOUNDED PRECEDING: Từ hàng đầu tiên của partition
- **n** PRECEDING: **n** hàng trước hàng hiện tại
- CURRENT ROW: Hàng hiện tại
- **n** FOLLOWING: **n** hàng sau hàng hiện tại
- UNBOUNDED FOLLOWING: Đến hàng cuối cùng của partition

ROWS - Tổng cộng dồn theo thời gian

```
SELECT
    employee, sale_date, amount,
    SUM(amount) OVER (
        PARTITION BY employee
        ORDER BY sale_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_total
FROM sales;
```

#	employee	sale_date	amount	running_total
1	Alice	2024-01-01	100	100
2	Alice	2024-01-02	200	300
3	Alice	2024-01-03	300	600
4	Alice	2024-01-04	400	1000
5	Bob	2024-01-01	150	150
6	Bob	2024-01-02	250	400
7	Bob	2024-01-03	350	750

ROWS - Trung bình hôm trước, hôm nay, hôm sau

```
SELECT
    employee, sale_date, amount,
    AVG(amount) OVER (
        PARTITION BY employee
        ORDER BY sale_date
        ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
    ) AS moving_avg
FROM sales;
```

#	employee	sale_date	amount	moving_avg
1	Alice	2024-01-01	100	150.0000
2	Alice	2024-01-02	200	200.0000
3	Alice	2024-01-03	300	300.0000
4	Alice	2024-01-04	400	350.0000
5	Bob	2024-01-01	150	200.0000
6	Bob	2024-01-02	250	250.0000
7	Bob	2024-01-03	350	300.0000

ROWS - FIRST_VALUE và LAST_VALUE

```
SELECT
    employee, sale_date, amount,
    FIRST_VALUE(amount) OVER (
        PARTITION BY employee
        ORDER BY sale_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS first_sale,
    LAST_VALUE(amount) OVER (
        PARTITION BY employee
        ORDER BY sale_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS last_sale
FROM sales;
```

#	employee	sale_date	amount	first_sale	last_sale
1	Alice	2024-01-01	100	100	400
2	Alice	2024-01-02	200	100	400
3	Alice	2024-01-03	300	100	400
4	Alice	2024-01-04	400	100	400
5	Bob	2024-01-01	150	150	350
6	Bob	2024-01-02	250	150	350
7	Bob	2024-01-03	350	150	350

ROWS - Tính lương so với trung bình phòng ban

```
SELECT
    name, department, salary,
    AVG(salary) OVER (PARTITION BY department) AS avg_dept_salary,
    salary - AVG(salary) OVER (PARTITION BY department) AS diff_from_avg,
    FORMAT((salary / AVG(salary) OVER (PARTITION BY department) * 100), 2) AS
percent_of_avg
FROM employees;
```

#	name	department	salary	avg_dept_salary	diff_from_avg	percent_of_avg
1	Frank	Finance	1100	1200.0000	-100.0000	91.67
2	Grace	Finance	1300	1200.0000	100.0000	108.33
3	David	HR	1500	1500.0000	0.0000	100.00
4	Eve	HR	1500	1500.0000	0.0000	100.00
5	Quang	HR	1400	1500.0000	-100.0000	93.33
6	Tuan	HR	1600	1500.0000	100.0000	106.67
7	Alice	IT	800	1000.0000	-200.0000	80.00
8	Bob	IT	1200	1000.0000	200.0000	120.00
9	Charlie	IT	1000	1000.0000	0.0000	100.00

RANGE - Frame Clause

```
PARTITION BY ...
```

```
ORDER BY ...
```

```
RANGE BETWEEN [start_point] AND [end_point]
```

- ❖ **RANGE** xác định frame dựa trên giá trị của cột được sắp xếp (logic range).
 - Tính toán dựa trên khoảng giá trị
 - Các hàng có cùng giá trị trên cột ORDER BY được xem như tương đương
 - Phù hợp khi tính toán trên khoảng giá trị, đặc biệt khi nhiều hàng có cùng giá trị

ROWS - Tính doanh thu 3 ngày gần nhất

```
SELECT
    employee, sale_date, amount,
    SUM(amount) OVER (
        PARTITION BY employee
        ORDER BY sale_date
        RANGE BETWEEN INTERVAL 2 DAY PRECEDING AND CURRENT ROW
    ) AS total_in_3_days
FROM sales;
```

#	employee	sale_date	amount	total_in_3_days
1	Alice	2024-01-01	100	100
2	Alice	2024-01-02	200	300
3	Alice	2024-01-03	300	600
4	Alice	2024-01-04	400	900
5	Bob	2024-01-01	150	150
6	Bob	2024-01-02	250	400
7	Bob	2024-01-03	350	750

ROWS - Tính lương so với trung bình phòng ban

```
SELECT
    employee, sale_date, amount,
    SUM(amount) OVER (
        PARTITION BY employee
        ORDER BY sale_date
        RANGE BETWEEN INTERVAL 2 DAY PRECEDING AND CURRENT ROW
    ) AS total_in_3_days
FROM sales;
```

#	employee	sale_date	amount	total_in_3_days
1	Alice	2024-01-01	100	100
2	Alice	2024-01-02	200	300
3	Alice	2024-01-03	300	600
4	Alice	2024-01-04	400	900
5	Bob	2024-01-01	150	150
6	Bob	2024-01-02	250	400
7	Bob	2024-01-03	350	750

Use case - Sử dụng Window Function

```
SELECT productId, productName, categoryId, Revenue
FROM (
    SELECT p.productId, P.productName, P.categoryId,
           SUM(OD.unitPrice * OD.quantity) AS Revenue,
           RANK() OVER (PARTITION BY P.categoryId ORDER BY SUM(OD.unitPrice * OD.quantity)
           DESC) AS rankInCategory
    FROM Product P
    JOIN OrderDetail OD ON P.productId = OD.productId
    GROUP BY P.productId, P.productName, P.categoryId
) AS RankedProducts
WHERE rankInCategory = 1
ORDER BY categoryId;
```

Bài tập Window Function

https://sql.zcode.vn/contests/CSDL_SQL/questions/9df170c05fc2482fb24874ef3a498087

https://sql.zcode.vn/contests/CSDL_SQL/questions/5068356730924ee3bdb211fac22a595e

https://sql.zcode.vn/contests/CSDL_SQL/questions/2e33a40949e04a858f7f9264d37780f0